nlcsplit: fragment and pair resolution of the VV10 nonlocal-correlation energy

Angran Xia*
School of Chemistry & Chemical Engineering, Shaanxi Normal University, Xi'an 710119, China
haangyeon@snnu.edu.cn

Changwei Wang
School of Chemistry & Chemical Engineering, Shaanxi Normal University, Xi'an 710119, China

Draft — 29 September 2026
ORCIDs PENDING, author-supplied

1 Abstract

2 Motivation and statement of need

Functionals that pair a range-separated hybrid with the VV10 nonlocal correlation term - ωB97X-V [1] and ωB97M-V [2] in particular - are a routine choice for non-covalent interactions. Their nonlocal contribution,

E_NL = $\frac{1}{2}$ ∫∫ ρ(r) ρ(r') Φ(r, r') d^3r d^3r',

is a two-centre sum, but what users get back is one number per grid point, or one total. When an interaction energy needs to be interpreted per fragment, the available options stop at the semi-local part: electrostatics, exchange, polarisation and charge transfer can be resolved by EDA schemes, whereas the nonlocal term is carried as an undivided block. The closest published treatment computes exactly that block as a whole and asks whether it is accurate [3]; the closest published explicit evaluation, JuNoLo [4], sums the double sum directly but targets an improved total energy and explicitly recommends itself for bonded systems "where one cannot tell apart fragments". Neither resolves the term into fragment pairs, which is what the literature screen of [S1] was run to find - and that screen's own record says the open-index *subject* search behind it is unreliable, so this is a screen, not an enumeration. That gap is what this package fills, and it is a capability gap rather than an accuracy gap: on the same grid and density our sum agrees with the native PySCF kernel to ~1e-8 Ha [S9].

3 Method overview

Kernel and convention. `w0 = sqrt(C|∇ρ|^4/ρ^4 + 4πρ/3)`, `κ = b(3π/2)(ρ/9π)^(1/6)`, `g = w0(r_i)R^2 + κ(r_i)` with same-centre pairing, `Φ = -3/(2 g g' (g + g'))`, and `E_NLC = ½∫∫ρρΦ + β∫ρ` with `β = (1/32)(3/b^2)^(3/4)`. Parameters are read from PySCF `nlc_coeff` rather than hard-coded. Same-centre pairing is the convention that reproduces the native kernel;

*Corresponding author; e-mail in the byline.

the cross-pairing variant `g = w0(r_j)R`^{2} `+` κ`(r_i)` is retained as a definition-sensitivity control and raises monomer energies by +0.1379 (water), +0.4182 (argon) and +0.1346 (methane) kcal/mol at grid level 1, and +0.9057 for benzene at level 0 [S5]. The offset is not a constant and it moves with the grid - the level-0 benzene figure is not the level-1 one - so these four numbers are one batch at the levels the producing script itself selects, and must not be mixed with another batch. [S5] now points at an archived run carrying all four, which it did not: an earlier version of this sentence gave benzene as 0.914 kcal/mol and labelled the whole list "at grid level 1", and neither value nor label was checkable, because the only carrier had been a gitignored working log. The rerun that closed the gap reproduced water and argon to the printed digit and gave benzene +0.9057 at level 0 - which is what the companion text already said, so the two papers had been disagreeing with each other about one number while citing the same source row - and added methane, which neither paper had mentioned.

Blocks and additivity. `pair_energy(A, B)` returns the half block $\frac{1}{2}$ Σ_i∈A Σ_j∈B; the inter-fragment term is reported as twice that, so `E_nl = Σ_A intra[A] + Σ_A<B inter[A,B]` holds as an identity. Measured residual: 1.73e-18 to 6.94e-18 Ha, over the five partitions and two repeated schemes of the archived level-1 water-dimer run [S2].

Why decomposing only the double sum is exact. β is a constant and $\int\rho$ is the electron number, so the local term $\beta\int\rho = \beta$N cancels identically from any superposition difference. Inter-fragment decomposition therefore needs no convention for splitting a local energy density: it is a property of the double sum alone [S3]. The flip side is reported as a limitation rather than hidden - the absolute local term of one fragment is exactly $\beta\cdot$N_A, so its scheme spread is β times the spread of the Becke population, 3.0415 kcal/mol per electron, i.e. 0.329 kcal/mol for the water dimer on the official S22 geometry and 0.300 on the constructed one (`nlcsplit/step_k_population.py`, whose two runs both assert that the fragment populations sum to the electron number).

Attribution. Four real-space partitions reachable through the PySCF grid machinery - Becke, Becke with the atomic-radius adjustment, Becke-Treutler and Stratmann - plus a nearest-atom hard assignment on a common grid. A point may be shared by more than one fragment; the factor matrix is exposed as `partition_factors(mol, level, scheme) -> (coords, base_w, S[N, natm])`, normalised so that `Σ_a S[i,a] = 1` pointwise. The function refuses to return unless two internal assertions pass: normalisation, and `base_w · S[i, owner(i)]` reproducing the weights that PySCF's own `get_partition` produces - an external anchor against the same C kernel PySCF uses, not against a re-implementation in this package. The two checks it enforces are pointwise normalisation `|Σ_a S[i,a] - 1| < 1e-12` and agreement with PySCF's own weights `max|base_w·S[i,owner(i)] - w_ref| < 1e-11`, with a normalised mean deviation below 1e-14; the tolerances are the assertion thresholds in the published test, not residuals we are claiming to have measured [S17].

Far-field cut. Points below `rho_floor = 1e-12` are dropped from the double sum. Sweeping 1e-8 … 1e-16 moves the inter-fragment term by at most 2.1e-11 Ha at grid level 1 and 4.2e-11 Ha at level 2 - 1.3e-8 and 2.6e-8 kcal/mol, which the run's own log prints as 0.000000 - while discarding 10.7-22.6% of points carrying ≤2.3e-6 electrons [S7], so the threshold is an implementation detail rather than a parameter. An earlier version of this sentence said "≤5e-11 kcal/mol": it had taken the archived figure, which is in hartree, and put a kilocalorie unit on it, claiming the term was about 260 times less sensitive to the floor than the archive shows. Both figures are negligible against the 1e-4 kcal/mol scale of the quantities here, so the conclusion survives the error, but the number did not - corrected against `evidence/recompute/20260928T150222Z_step_d_floor.out`, whose header already records the units.

4 Software description

nlcsplit/ nlc.py VV10 fields, pair_energy (gemm path), fragment_decomposition partition.py four attribution schemes, partition_matrix, partition_factors geomlib.py single source of geometry truth; internal-coordinate assertions at construction; covalent-connectivity fragment splitting; official GMTKN55/S22 loader step*.py one script per table in the paper; stdout archived UTC-stamped figures.py figures built from the JSON sidecar, never from log text tests/ kernel, regression, geometry and partition-factor suites examples/ quickstart plus two comparison checks

Dependencies: NumPy and PySCF. `partition.py` reaches into `libdft.VXCgen_grid` through ctypes deliberately - it is the entry point that `gen_grid.get_partition` itself uses, which is what makes the audit above external rather than circular.

Representative output, produced by `nlcsplit/examples/quickstart.py` (ωB97X-V/6-31G*, water dimer, grid level 1, kcal/mol; archived verbatim with its interpreter and module fingerprints as `evidence/release_qa/20261003T1901Z_quickstart_readme_example.out`):

inter[(0,1)] = -0.5237 intra = -3.5457 -3.7437 E_nl = -7.8130 E_loc = 60.8303 E_total = 53.0172

`E_nl` equals intra + intra + inter as required, and `E_total` may be compared with the native kernel evaluated on its own reference grid, which gives 53.017 and 53.016 for this system and level (difference +0.002 kcal/mol, which is the grid representation difference of section 3.1 of the companion text rather than an algorithmic one) [S9].

5 Impact

The package makes three things checkable that previously were not. It gives a number for how much of the nonlocal correlation is inter-fragment, which on the six S22 complexes that carry a counterpoise reference at grid level 2 accounts for 0.92-1.08 of the frozen-density superposition difference - association-time nonlocal correlation is almost entirely carried by point pairs straddling the fragments [S2, S10]. It supplies the per-system uncertainty that the attribution choice contributes (0.004-0.247 kcal/mol absolute, 0.8-18.2% relative, with no ordering by interaction type or by system size), which is the error bar any future use of such numbers must carry [S12]. That span is not basis-set invariant: on the water dimer it is 0.0249 kcal/mol in STO-3G against 0.0382-0.0400 in the four larger sets, so the number moves by ~38% between STO-3G and 6-31G* while the four sets beyond STO-3G agree with each other within 5% [S12]. Nor is it grid-converged to better than it is printed: level-2 values are stored to three decimals, so a level-1 to level-2 change below 5e-4 kcal/mol is not resolvable in the archived table, and the one four-decimal ladder we ran gives 0.0002 kcal/mol between levels 1 and 2 - small, but not zero. And it produces evidence about the quantity itself: where the share is defined at all it ranges from 16% of ΔE_xc (water) to 97% and 52% for the two geometries of the benzene dimer, and on the methane dimer the two parts carry opposite signs so the package's own rule ($|\Delta$E_NLC/ΔE_xc$| > 2 \Rightarrow$ report the two absolute terms, not a ratio) refuses to state a share at all. The VV10 term therefore cannot be reported as "the dispersion energy" without qualification, and on one row of our own table not even as a percentage [S2].

6 Availability and quality assurance

Apache-2.0; no GAMESS source is included, linked or required. That claim is checked by scanning the published tree for GAMESS *source artefacts* specifically - `find` over `*.src`, `*.f`, `*.for` and `GAMESS*` returns 0 - and not for the word. An earlier version of this sentence asserted that the

word appears only in the package docstring, and that assertion was false: 13 files contained it, 8 of them as part of a working-machine path (`/root/gamess_work/...`, `/mnt/e/Project/GAMESS/...`) including inside the shipped `step_h.json` and `run_all.sh`, which is how a repository name that reads as "this is the GAMESS project" got into a release that claims not to contain it. Two defects were behind that: the gate's patterns covered artefact extensions but not path forms, and it iterated the list of copied names rather than the directory tree, so 13 `.pyc` files that pytest had left inside the release directory were never opened at all. Both are now fixed - the gate walks the tree and fails by default on path forms, bare cluster addresses and account names - and the published copies are scrubbed of those paths while the numbers are left untouched, with a byte-level ledger of every replacement kept beside the release and the archived stdout in this repository still hashing to what was recorded for it. After the fix, counted where the count belongs - over the exported tree (dist/nlcsplit, which is build output and is not itself tracked), text files and exact case - the word appears in 9 files and the working-machine path forms in 0. The 9 are the disclaimer itself (the package docstring, `README.md`), prose about the disclaimer (`CONTRIBUTING.md`, the published docs/tables_and_provenance.md, `.github/workflows/ci.yml`), the two scanners' own detection patterns (`tools/audit_no_gamess.js`, `tools/make_public_repo.js`) and two archived scanner stdout files under `evidence/release_qa/`. Counting case-insensitively gives 13: the four extra are `EXPORT-MANIFEST.txt`, the release-QA README and its own manifest, and the gate self-test record, each hit being a file-name reference to the scanner rather than a new occurrence of the claim. Compiled residue is outside both counts - a `.pyc` carries the same strings in compiled form, the export manifest lists none, and gate R fails a build containing any, which is not a hypothetical: a later pytest run has left residue of exactly that kind inside the export directory on this machine, and the next rebuild has to clear it. That is a count of a tree, and it is not what the shipped scanner does: it refuses to read a tree at all (`-no-install` is rejected outright, because a stale copy of the build directory is how an earlier version of this check passed), installs into a throwaway target, and walks what lands in site-packages, where the occurrences it permits are the exemptions it prints by name. The archived run of that scan (`evidence/release_qa/20260929T213552Z_no_gamess_scan_after_allow.out`) reports 45 installed files, three named exemptions, 0 GAMESS hits outside the exemption ledger and 0 licence-conflict signals at rc=0. Only two of the three exemptions are the licence disclaimer - the docstring and the same sentence as rendered into the wheel's METADATA; the third is pip's own `direct_url.json`, discussed below. The remote copy is not counted a third time here: it is checked blob against blob by `tools/publish_github.js` in its verify-only mode, and the one push on record matched 121 local blobs to 121 remote ones with nothing unmatched on either side (`evidence/release_qa/publish_record_20260930.txt`). Two further defects were found while re-checking this section on 2026-09-29, and both were of the kind a source-tree reading cannot see. The first: an earlier build was reported fixed and still shipped two script-style `test_*.py` modules at package top level; the wheel then had 36 members and none of them was a script-style test - it grew again when `step_l_sweep_compare.py` was added to the whitelist, see below. That count is a dated reading, not a current one: the census now run by `evidence/release_qa/wheel_census.py` reports 38 members - 7 test modules, the 2 non-test files that live under `nlcsplit/tests/`, and 29 others - with the same verdict that no script-style test is shipped [`evidence/release_qa/20261004T0317Z_wheel_census.out`], and it prints the four categories alongside their sum so a silently dropped category cannot pass as a member count. The second is worse and was found only by looking at the artefact: **`fastpair.py` was not in the package at all**, while the shipped `step_j_scaling.py` imports it - so a reader installing the wheel could not run a single number from the companion algorithm paper, and gate E (which checks that every import is *declared*) stayed silent because the import is intra-package. `fastpair.py` and its test module are now in the manifest. In the same pass `test_tolerance_probe.py` was published

too, because a tolerance tightened from 0.10 to 1e-4 kcal/mol is a claim about measured spread and the measurement belongs beside it.

A third omission of the same kind was found by asking the question the other way round: `run_all.sh` - the advertised one-command reproduction path - knew four of the thirteen `step_*.py` scripts, and `step_l_sweep_compare.py` was missing from the shipped step list altogether. That script is the only producer of the "agrees to 2e-3 kcal/mol with an independently written script" sentence in section 2.1, so until now the published tree contained neither that script nor any artefact carrying the comparison column. The driver now lists all thirteen with each one's precondition (which need an argument, which need a geometry download, which need the node contention-gated - the timing script counts external runnable tasks and marks a dirty sample unusable rather than trusting the wall clock), derives the repository root from its own location instead of a hard-coded working path, and refuses to run the argument-taking ones rather than silently running them at their defaults.

That repair left no guard behind, and the omission came back the only way such a thing comes back: a fourteenth script. `step_m_ccsdt.py`, the producer of the coupled-cluster anchor, was added to the package on 2026-09-30 and never reached `run_all.sh`, so for two days the published tree again held a table-producing script that no entry point could run. It was found on 2026-10-01 by enumerating the shipped `step_*.py` files against the driver's command table, and that comparison is now an export gate of its own (`gateX_runner_coverage`) with the self-test the earlier repair lacked: truncating the command table to the four entries of the original bug must make it report ten missing scripts, and a table completed by construction must make it silent. It is evaluated against the export tree rather than against this repository, because the copy that has to be runnable is the reader's.

The suite collects 79 tests in the working repository - 58 under `-m "not slow"` and 21 under `-m slow`, a partition of the same collection and not two numbers to be added (`evidence/release_qa/20261009T105424Z_test_count_census.out`, whose per-module rows must sum to the collected total and which also groups them by what the wheel actually carries). All 79 have been executed on the tree those counts describe: 79 passed with nothing skipped and nothing deselected in 504.41 s, with the run's exit code kept in a file of its own beside the log (`evidence/release_qa/20261009T094106Z_full_suite_79passed.log`, `evidence/release_qa/20261009T094106Z_full_suite_79passed.log.rc`) [S19]. The five tests added since the previous census are `nlcsplit/tests/test_nlc_closure.py` - the closure guard that used to be an identity, its monolithic reference, and the two regroupings that must make it fail. They ship in the wheel; until today they did not, and what changed is the export gate's own complaint rather than a decision by us (`pip install .` would install the file, the wheel had no stated reason to omit it, and it was not named in the list of deliberate exclusions). That is recorded here because a guard a reviewer cannot run from the installed package is not evidence of anything to that reviewer. That run used `OMP_NUM_THREADS=4` where the run it replaces used the machine default, so its wall clock is the longer of the two and the comparison worth drawing is the count, not the timing. The superseded run is kept beside it: 74 passed with nothing skipped in 328.87 s on the working tree of 2026-10-05, whose uncommitted files the archive itself lists rather than asserting a clean tree (`evidence/release_qa/20261005T0512Z_suite74_after_figure_fixes.out`, which supersedes a run of the same size taken 43 minutes earlier on an intermediate tree; its census is `evidence/release_qa/20261005T0404Z_test_count_census.out`). The previous full run, on the 69-test tree that carried the convention guard added last, was 69 passed with nothing skipped in 380.52 s (`evidence/release_qa/20260930T233800Z_suite69_full.out`). An earlier run of the first 65 of them - 65 passed, nothing skipped, 710.73 s, with the three collection counts and the Python, PySCF, NumPy and SciPy versions in its own archived log (`evidence/release_qa/20260930T202235Z_suite65_full_and_counts.out`) -

is kept beside it, and the useful fact is that the two disagree on wall clock by 1.87x while the added tests take 0.28 s collectively: that is machine load, not content, and it is why no figure here is called "the suite time". Between those two runs an intermediate artefact (`...20260930T230121Z_suite69_fast_and_diffsurface.out`) recorded a partial run with the slow 20 carried over on a change-surface argument; the full run above replaced it, and the argument survives only as a worked example of when a carry-over would have been legitimate. That execution is of the working tree under `PYTHONPATH=.`, and the log's own header prints `git status` - the 65-test one listed uncommitted changes - so it establishes the floor for the code as it then stood and says nothing about what an install ships. The out-of-tree material below belongs to objects smaller than the working tree. The published wheel ships eight test modules plus `__init__.py` and `conftest.py`; those eight collect 59 in today's tree, and the 20 that stay behind are exactly `test_fastpair_ide.py`'s (both counts come from `evidence/release_qa/test_count_census.py`, which groups the modules by the wheel's own member list and refuses to print unless the two groups sum to the collected total [`evidence/release_qa/20261009T113507Z_test_count_census.out`]) - the peer's frozen control module, whose `fastpair_ide.py` is deliberately not distributed, since shipping the test without the module would be a collection-time ImportError. One printout taken twelve minutes before that one (`evidence/release_qa/20261009T113331Z_test_count_census.out`) says seven modules and 54, and it is kept as the record of a mistake rather than deleted: the script had been choosing its wheel with `sorted(os.listdir("dist"))[0]`, and minutes earlier I renamed the previous build to a name prefixed `archive-` inside dist/ precisely so it would survive as evidence that a rebuild had happened - "a" sorts before "n", so the census quietly counted the old wheel's members and returned numbers that looked entirely plausible because they had been true on 2026-10-05. It now insists on the canonical filename and exits if that file is absent. Two different objects must not be quoted as one: `pip install .` from the source directory in a clean environment collected 63 in the archived logs quoted below, because that install pulls in the whole tree including the control module, and its archived full run is 62 passed and one skipped (the S22 geometry fetch, by design) in 1739.26 s, whose `-m "not slow"` subset is on record twice at 42 passed and one skipped and 20 deselected, in 10.78 s and in 11.56 s. That collection size is not a constant either: re-running the same two shapes on 2026-10-01, after four tests had been added, gives 69 collected - the in-tree step 2 of the workflow 69 passed with nothing skipped in 576.08 s under PySCF 2.14.0 and in 673.47 s under 2.12.1, and the out-of-tree step 3 68 passed and one skipped in 824.43 s and 545.01 s respectively, the skip again being the S22 geometry fetch, which is only reachable from outside the working tree because the downloaded geometry is not part of the install. Both matrix arms were therefore run locally before the remote exists, which is the point of the dry-run - and the prediction it falsified was ours: no `nlc_coeff` or `get_partition` API drift between the two versions showed up at all (`evidence/release_qa/`). `pytest -pyargs nlcsplit` against the built wheel, run from a directory that is not the source tree, exercises only the published tests. The current wheel was re-measured this way on 2026-10-02, in a distribution installed outside the tree with `pip install -no-deps -target` and the tests started from a different directory: 48 passed and one skipped in 361.28 s, exit status 0, with the wheel hash recorded in the archive (`evidence/release_qa/20261002T0815Z_wheel_install_pytest_out_of_tree.txt`). It was re-measured again on 2026-10-03, after the convention tests had grown: 50 passed and two skipped in 308.95 s, exit 0, and both skips are the official-geometry tests, which cannot run in an install that does not carry the downloaded geometries - they skip rather than pass silently, and the reason is printed (`evidence/release_qa/20261003T0955Z_wheel_and_repo_pytest.txt`). And twice on 2026-10-05, against the wheels rebuilt during and after the re-audit: 51 passed and three skipped in 306.16 s against `9ecfc5ee...` (`evidence/release_qa/20261005T0446Z_wheel_out_of_tree.out`), then the same 51 + 3 in 304.19 s against the build of ten minutes later, `ac354030...` (`evidence/release_qa/`

20261005T0506Z_wheel_out_of_tree_current.out). Neither was a wheel this paper ships, because the export rebuilds it; the build measured today is 7bdcb6f6... - 56 passed and 3 skipped in 404.44 s, exit 0, with its hash, its size and the git commit it was built from recorded in that log's own header (evidence/release_qa/20261009T113900Z_wheel_out_of_tree.out, the exit code in evidence/release_qa/20261009T113900Z_wheel_out_of_tree.out.rc). Measuring a container that the next export replaces is only worth printing if the content is pinned, and here the pinning is a comparison rather than a sentence: against the build produced by the next run of the export, evidence/release_qa/20261009T120455Z_wheel_member_diff.out diffs the per-member digests of the measured build - 39 members each, 37 digests agreeing, all 34 Python modules under the package directory byte-identical, the two exceptions being METADATA and the RECORD that lists METADATA's own digest. METADATA moves for a reason worth stating plainly: pyproject.toml embeds README.md into it, so any wheel hash written inside the README is necessarily the hash of a build made before that sentence was written. The paragraph on the 2026-09-30 rebuilds further down gives the other mechanism by which container hashes move even when no packaged byte does. The isolation claim here is weaker than the 2026-10-05 one and the archive says so instead of glossing it: python3 -m venv now fails on this machine for want of ensurepip, its error printed verbatim in the same file, so the tests ran from /tmp with PYTHONPATH pointing only at a pip install -target directory. That establishes that the wheel's own contents run where the repository is not on sys.path; it does not establish that they coexist with arbitrary other packages in a fresh interpreter. The third skip in both is the new geometry-label test, for the same reason as the other two; and the The 2026-10-05 number supersedes the older, smaller-wheel figures below only for the question "what does today's install pass"; the older logs stay because each is quoted at its own collection size. Every figure below is quoted at the collection size of the log it is taken from, and none of them is a sum of the others: the 43-test wheel is on record at 42 passed and one skipped, twice, at 580.82 s and at 406.75 s, and its -m "not slow" subset at 24 passed and one skipped and 18 deselected in 4.63 s. The 44th published test - the soft-attribution check against an analytic Gaussian, added in the 2026-10-01 commit 647a280 - postdates those three logs and predates the two that follow it, which are runs of a 44-test wheel and each name the wheel it tested: 43 passed and one skipped in 655.52 s against d175c7b8..., then 43 passed and one skipped in 519.36 s against a 36-member build recorded as d123d1c8... in its own log. The 45th, the grid-measure check test_base_w_on_real_density_is_not_a_measure_at_all, is newer than those logs and has since been run: rebuilding the export produced 149e727b..., whose out-of-tree -pyargs run collects 45 (27 under not slow, 18 under slow) and reports 44 passed and one skipped in 322.21 s. That timing is not comparable with the two above it - both earlier commands are recorded with OMP_NUM_THREADS=2 in their headers, this one left the thread variables unset on a 20-core box - which is exactly why the counts are quoted per log and the wall clocks are never pooled. Adding the coupled-cluster script to the packaging list rebuilt the wheel once more (7e7f9d71..., 37 members, the eight test files byte-identical to the 36-member build), and that wheel was also tested out of tree: 44 passed and one skipped again, in 504.67 s. The 1.57x spread between the two runs on the same box, with the same unset thread variables, on wheels whose tests are identical, has no cause recorded in either log - which is the strongest argument here for quoting timings per artefact rather than reducing them to one number. A third rebuild followed the frozen-core refactor of the coupled-cluster script, which also shipped the convention guard as the wheel's seventh test module (55033716..., 38 members, collecting 49): its 31-test fast subset was run out of tree - 30 passed and one skipped in 25.72 s - and its 18 slow tests were deliberately not, because the eight test files they live in and every library module they import are byte-identical to the 37-member wheel that was run in full nine minutes earlier. That is written into evidence/release_qa/20260930T231147Z_wheel_pyargs_49_member38_fast.out as a

carried-over result with the check that licenses it, rather than presented as a run that happened. Naming the hash rather than claiming "the current wheel" is the point - every export rebuilds the wheel, so its SHA-256 changes even when no packaged byte does, and the two runs recorded nine minutes apart on 2026-09-30 name two different hashes. The mechanism was measured rather than assumed, and it is worth knowing precisely because the naive version of the claim is wrong: four rebuilds of a tree whose `nlcsplit/` and `nlcsplit/tests/` bytes never moved gave three distinct hashes, two of the four builds being byte-identical to their neighbour - the zip entries carry the build time at two-second resolution, so builds inside one tick reproduce and builds across a tick do not. What never moved is the member list: 38 members in each build, with the wheel's per-member digest list (its `RECORD` file) comparing clean line by line across builds. So `RECORD` identifies the software and the artifact hash identifies the build, and this paragraph quotes them apart for that reason (`evidence/release_qa/20261001T053741Z_wheel_hash_is_not_content.txt`). All of it, including the shell-quoting bug that left the hash field blank in one of the two older logs, is in `evidence/release_qa/` [S19]. Tables map to one `step_*.py` each; the archived stdout a claim cites lives under `evidence/**` with a UTC-stamped filename and a hash manifest, while working logs under `nlcsplit/logs/` are gitignored scratch and are not cited as evidence. Including a naive triple-loop versus BLAS comparison of the kernel, per-scheme additivity checks, and geometry guards that assert bond lengths and angles at construction time.

Fifteen export gates run locally on every rebuild (`node tools/make_public_repo.js`, which ships in the published tree). They are not all of equal standing, and the difference matters: some exist because they caught a real defect, others only because we could not otherwise show the earlier ones were not vacuous. The last column says which, and "planted" means the gate has never fired on real input - it fires on a fault we put in deliberately to prove it would.

Gate D exists because the first three could only fail in one direction. A, B, C catch material that should not be present; what actually went wrong repeatedly was material that should have been present and was not - a cited stdout living only on a remote machine, calibration scripts inside a gitignored directory. Those produce a clean export and a reader holding a citation to nothing. Gate E has the same blind spot in the other direction and we hit it: it checks that an import is *declared*, so it says nothing when a shipped module imports a sibling that was never packaged - the `fastpair.py` defect above.

The gate code itself now ships, and that exposed a self-reference. `tools/make_public_repo.js` and `tools/audit_no_gamess.js` are published under `tools/` because this section and `CONTRIBUTING.md` tell the reader to run them; until 2026-09-30 they were not, so that instruction was executable only on the authors' machine - the same class of defect as a citation pointing into a gitignored directory. Publishing a scanner, however, publishes what it scans for: the exemption ledger printed by the first build that contained the scanner listed 54 lines, and every one of them was a detection pattern naming a cluster internal address, an SSH port, an account name or a working directory. Those ten rules, together with the fault fixtures that exercise them, now live in `tools/intranet_patterns.private.js`, which is excluded by design and named in the export report. The published copy keeps the shape-based rules (credential assignment, private-key block, `ssh ... @`, bare-IP user), loads the named rules when they are present, and prints `note_private_rules_loaded`; a run that cannot load them is not reported as clean - it says so, and gate B is then checking four rules rather than fifteen. Running the published copy against a copy of the published tree turned up two more holes of the same kind: rebuilding crashed on the two appendix sources that are renamed on the way out, and the proxy scrub was not idempotent, stacking its own replacement comment a second time. Both are fixed, and a second run over the same tree reproduces the first byte-for-byte except for the export timestamp (`evidence/release_qa/`) [S19]. `tools/audit_no_gamess.js` makes its single legal GAMESS hit a

gate	what it checks	standing
R	walks the **export tree** for build and pytest residue left inside it - `__pycache__` and `.pyc`, `.egg-info`, a stray `setup.cfg` - which is how a compiled docstring carries a repository path out even when every source file is clean	real defect: 13 `.pyc` files pytest had left inside the release directory, never opened by the old check, which iterated the list of copied names instead of walking the tree
A	no GAMESS source artefacts (matched on source patterns, not on the word "gamess", which the disclaimer itself contains)	real defect: a permanent false positive on our own disclaimer, which is why it matches patterns and not the word
B	walks the **published tree** and fails by default on credentials and internal-network strings - private-key blocks, password assignments, `ssh ... @` targets, bare-IP users - and, through the ten named rules that stay out of the release, on working-machine paths, bare cluster addresses and account names	real defect: the 13 published files that carried `/root/gamess_work/...` and `/mnt/e/Project/GAMESS/...` while the gate reported every one of its arrays empty, because its patterns covered artefact extensions but not path forms
C	every `evidence/*/MANIFEST.sha256` reproduces byte-for-byte	real defect: a Windows path making `sha256sum` report six mismatches that were not there
C2	the scrubbing changed no digit in any number, **and** every file in a published evidence directory is registered in that directory's own source manifest	the second half fired on real input the day it was written: a comparison JSON produced by a step script landed in `evidence/agentb_sweep/` with no manifest entry, and the build refused to ship it (`shipped: false`)
D	every path cited by the companion text's provenance appendix is actually shipped	real defect: three citations that resolved to nothing in the published tree
E	every import in a shipped module is declared in `pyproject.toml`	real defect: `figures.py` importing matplotlib, which no dependency listed
F	the wheel and sdist member lists are what we think they are	real defect: two script-style tests that survived a build already reported fixed
G	`LICENSE` in the **source** file carries the real copyright line, not the Apache placeholder	real defect: the legal statement existed only in the generated copy; fires today on a restored placeholder
T	nothing in the export tree is missing from the export manifest	added as a belt-and-braces check; no firing on real input is recorded in this repository, only on the faults we planted
U	every module a shipped file imports from `nlcsplit` (absolute, dotted or package-relative) is itself in the shipped tree; the gate also self-tests against a planted missing sibling, and a build where that self-test does not fire does not ship	real defect: it was written for `fastpair.py`, which gate E could not see because an intra-package import is not a declared dependency
V	every Python module inside a packaged directory is either in the export whitelist or named, with a reason, as an unreleased internal module - this is the set `pip install .` actually puts in `site-packages`, which the whitelist does not constrain; self-tests fire on a planted stray and stay silent on the named ones	real defect: found by the shipped licence scanner, which walked 45 installed files from a source-directory install and found the peer implementation among them
W	no manuscript or README quotes a *share* for a system whose denominator cancels, i.e. where `abs(dE_NLC/dE_xc) > 2` - the same rule `step_f_dispersion.py` applies to its own output, extended to the prose	real defect: three of them on the day it was written (the companion text's conclusion, this paper's section 4, and a line the author had just added to this README), all quoting a share equivalent to a ratio above 2 that section 3.2 of the same paper argues against - and the gate caught its own author reintroducing the number while documenting the gate
X	every `step_*.py` in the export tree appears in `run_all.sh`'s command table, so each table in the text has an entry point a reader can actually invoke	real defect: `step_m_ccsdt.py` reached the release on 2026-09-30 without ever being registered, which is the same omission the driver had been rewritten to fix - that rewrite left no guard, and this gate is the guard it lacked. The self-test truncates the command table to the four entries of the original bug and must then report ten missing scripts, and must go silent on a table completed by construction
Y	every image and link target in	planted only: it has not fired on real input, its

named rule in which the file *and* the line must match, so a non-disclaimer sentence inside the same file is still a violation, and it refuses to scan a directory that was not produced by a real install, since reading a stale copy of the build directory is how an earlier version of this check passed.

Installing from the source tree is not the same act as installing the release, and we had been describing it as if it were. `pip install .` collects every module inside the package directories; the export whitelist does not constrain it. An install from the source directory collects 37 packaged modules where the wheel ships 31 (the scanner walked 45 installed files in total), and among them it reported `nlcsplit/fastpair_ide.py` - the peer implementation this repository keeps as a frozen comparison, deliberately not released - sitting in `site-packages`. Gate V now fails a build if any Python module inside a packaged directory is neither whitelisted nor named, with a stated reason, as an unreleased internal module; the six currently named are that implementation, its test and its validation script, plus three measurement scripts whose outputs are archived under `evidence/`. The same run also surfaced the provenance record pip writes into its own `dist-info` directory, which stores the *caller's* install path - here a working-machine directory whose name contains the very word being scanned for. It is a named exemption whose predicate strips the URL value before deciding, so anything other than a path inside that file is still a violation.

Release hygiene is now executed rather than specified: a clean-environment install of the source tree, and an out-of-tree `pytest -pyargs nlcsplit` against the built wheel from a directory outside the repository, both recorded with the wheel hash. Writing the specification down is what produced two of the fixes: script-style `test_*.py` modules were removed from the package (pytest collects and executes them at import), and the `slow` marker was registered in package configuration rather than only in repository configuration. An earlier out-of-tree import by path - weaker, since it read the exported tree rather than an installed distribution - found that `step_i_gridrepr.py` ran a full self-consistent-field sweep at import time because it had no `__main__` guard, since fixed.

The continuous-integration workflow has now been observed executing, and what it showed was not the thing we had written down. Recording the workflow had already produced one real fix: its PySCF version matrix interpolated as `pip install "pyscf==>=2.13"`, which is not a valid requirement, and that class of defect only an executed run catches. What we had reported in place of an execution was the weaker claim **no CI run has ever been observed**, and it was wrong. Asking the API with credentials returned five runs on `main`: #1 (`0b903ab`) and #2 (`99584d4`) completed `failure` in both matrix arms, #3-#5 completed `success`, #5 being the current head. The earlier look was made without credentials at a repository that was then private, and that request cannot return a run list at all - it is a method that produces silence, which we had read as absence. It is the same failure shape as the withdrawn ORCA agreement recorded later in this section: the label we searched for was not the label that exists.

Both failures sat in one step, `Run node tools/make_public_repo.js`, and had one cause. That step is a shipping decision rather than a check on already-shipped content, and its share-rule gate spawns `tools/audit_share_rule.js`, which the release list did not carry. Absent that file, the gate's bidirectional self-test returns non-zero for the deliberately wrong sample *and* for the correct one, so the gate does the right thing - refuse to ship on a self-test it cannot trust - and reddens every push while nothing in the chemistry is wrong. Two changes follow, both in the shipped tree. The scanner ships now, and its default document list falls back to the material a published copy actually contains, printing that it fell back; falling back does not relax the rule, because an unreadable file still fails and a denominator of zero also fails. And the workflow no longer runs the exporter, keeping the two steps that can hold on a published tree. The archived observation is `evidence/release_qa/20261001T150706Z_github_actions_first_runs.out`: run, job and step fields as the API returns them, plus the pytest summary lines verbatim, because a step reported `success` with no count is the same void as the 404 we had mistaken for one. On both arms the fast lane reads

30 passed, 1 skipped, 18 deselected, and the full and out-of-tree runs each read 48 passed, 1 skipped - those are the CI arms' own collection sizes and cannot be re-measured without a push. What was re-measured locally on 2026-10-03 is the working repository: its fast lane collects 50 passed and 22 deselected in 26.5 s, its full run 72 passed in 350 s, and the installed wheel 50 passed and 2 skipped. That is 52 items, not the 72 the working repository collects: `test_fastpair_ide.py` is not distributed, because the control module it imports is not distributed either.

The fifteen shipping gates still run in the working repository before a release, with their private rules module and their manuscripts present, and CI does not vouch for them - it cannot, since four of the fifteen read material that is deliberately not published.

Independent verification of the arithmetic now has a second program behind it. An earlier version of this documentation withdrew an ORCA agreement because re-reading the archived output found no separately reported nonlocal term - and that withdrawal was itself wrong: the quantity is printed, under the label `NL Energy, E(C,NL)`. Grepping the same output for "nonlocal" or "VV10" is not a way to find it, but not for the reason we first gave: the words are there, in the run banner's feature list (`NL-DFT gradient (VV10)`, line 116 of `evidence/orca_run/h2o_dimer.out`), which prints on every run and reports no number. The `E(VV10-nl)` column that does match belongs to the PT2 correlation of double-hybrid functionals, which is a different physical quantity. With the label known, twenty-four ORCA 6.0.1 single points covering all eight of the reported systems agree with ΔE_NLC to within 1% of the term everywhere (0.0004 kcal/mol on the water dimer up to 0.0416 kcal/mol on the benzene dimer A), with the outputs and their SHA-256 manifests in the repository [S18]; the comparison is regenerated from those artefacts by `evidence/orca_widen/generator_cross_program_table.py`, which refuses to emit the table if the system list or any row's basis, grid level or counterpoise setting disagrees. A second program can check what the term is worth; it cannot check who owns it, because it does not attribute. That gap is now measured too, in `evidence/attribution_referee/`: on a two-Gaussian density an exact reference cross term is built that contains no partition, no grid assignment and no fragment ownership anywhere in its definition, and evaluated three independent ways - a 3-D Cartesian tensor product, a 5-D cylindrical reduction about the internuclear axis, and importance-sampled Monte-Carlo with its own standard error. Handed the exact density factors, the package's own summation code reproduces that reference to `7.96e-07` Ha worst, inside the reference's combined measured error bar of `1.060e-05` Ha; handed the production Becke/owner assignment, the same code sits `1.7e-08` to `1.26e-04` Ha away from it, and the offset does not move when the node set is changed, so it belongs to the convention rather than to the grid [S21]. The referee states its own resolution rather than hiding it: the offset exceeds the error bar in 5 of its 12 configurations, and in the other 7 it is smaller than the reference's uncertainty, which is reported as "cannot distinguish", not as "agrees". Two limitations there are unchanged. A third, fully independent implementation (Psi4, or libxc's `xc_nlc_vxc` through its own bindings) is still outstanding - the point is that ORCA is the *only* second program, not that there are three. And the scope of what is checked: on the water dimer the comparison covers the absolute term for the complex and for both monomers, while for the other seven systems the second program checks the superposition difference - which is the quantity every published claim here is made of, but not each absolute per-fragment value.

7 Limitations

Quadratic cost in retained grid points, with no sub-quadratic approximation. One attempt at a certified screening was built and measured, and rejected - not on the weak version of the argument we first wrote here. The earlier sentence claimed the bound prunes nothing "at the tolerance the

companion text needs (1e-8 Ha absolute)"; that mis-states both sides. `1e-8 Ha` is a per-node-pair acceptance constant, and the companion text needs 1e-6 kcal/mol on a fragment-pair term, i.e. 1e-6/627.509474 Ha in the same unit - so the constant and the requirement were never the same kind of quantity. Measured properly, three things hold: at atom-block granularity the *smallest* block-pair bound in a real water dimer is 197.96 Ha against a quantity of -8.3e-4 Ha, so nothing is prunable at that granularity whatever the tolerance (0 of 21 pairs, and 0 of 300 on the 24-atom sandwich); refining below the atom block does fire, and at the accuracy the decomposition needs it still evaluates 84.4-95.4% of the far-field point pairs; and with no other runnable user task at the sampling instant (a contention gate, not a claim that the machine was idle), the screened path is slower than the dense sum in every one of 33 timed screened/exact pairs across two independent windows on the same box - 1.08-10.28x in the archived window and 1.37-13.29x in the second (1.95-10.28x and 2.63-13.29x at the tightest tolerance tested), with no faster sample in either. The radius the bound subtracts is set by the far tail of each atom's density, so the certified separation comes out zero for every pair, including the ones facing each other across the inter-fragment gap. That is a statement about this particular bound, not about screening in general, and the measurements are in `paper/p2.md` with their artefacts under `evidence/bound_forms/`, `evidence/fastpair_ab/` and `evidence/scaling/` rather than quietly dropped [S20]. Interpretation limited to the two-body and fragment-pair level. Results depend on the choice of real-space partition to the extent quantified in section 4. VV10 only - the τ-dependent rVV10 form is not implemented. And it is not an energy decomposition analysis: it does not separate electrostatics, exchange-repulsion, polarisation or charge transfer.

8 Appendix: sources

Paths are relative to the repository root, because that is what the export gate resolves them against. Every path below is tracked by git, and the build refuses to produce `software.tex` if one is not. Three rows used to name `logs/<something>.out`, which is not a path in this repository: `nlcsplit/logs/*.out` is gitignored scratch, and those files are tracked only as the hash-manifested copies under `evidence/**` (the build does place a scrubbed copy of two of them inside the package, but a citation should point at one tracked original, not at a generated duplicate). The two `.json` logs in the S2 row are tracked in place, and are cited as they are kept.

Literature is not in this table. Where a sentence needs a published work it uses a `[[key]]` citation resolved against `paper/references.bib`, whose entries were reverse-checked through Crossref by DOI; the queries for the software and benchmark entries are archived in `evidence/bib_provenance/G_s22_provenance_20260930.md`.

References

[1] Narbe Mardirossian and Martin Head-Gordon. ωb97x-v: A 10-parameter, range-separated hybrid, generalized gradient approximation density functional with nonlocal correlation, designed by a survival-of-the-fittest strategy. *Physical Chemistry Chemical Physics*, 16:9904, 2014. doi: 10.1039/c3cp54374a.

[2] Narbe Mardirossian and Martin Head-Gordon. ω b97m-v: A combinatorially optimized, range-separated hybrid, meta-gga density functional with vv10 nonlocal correlation. *The Journal of Chemical Physics*, 144:214110, 2016. doi: 10.1063/1.4952647.

[3] Saswata Dasgupta, Etienne Palos, Yuanhui Pan, and Francesco Paesani. Balance between physical interpretability and energetic predictability in widely used dispersion-corrected density

Tag	Artefact	What it fixes
S1	`RESEARCH-20260929-lit.md` §1, §6, §9	the literature screen behind the statement-of-need comparison, together with that file's own measurement that open-index *subject* search recall is poor — which is why §1 calls the gap a screen and not an enumeration
S2	`evidence/recompute/20260929T010500Z_step1_geomfix.out`, `evidence/recompute/MANIFEST.sha256`, `nlcsplit/logs/step_h.json`, `nlcsplit/logs/step_h_l2cp.json`, `MAIN-skeleton.md` (its uncertainty-decomposition, P_frz, and S22-subset tables)	the per-scheme block-sum residual - 1.73e-18 to 6.94e-18 Ha, printed for five partitions and two repeated schemes of the level-1 water dimer at lines 6-14 of the first artefact - the only measured additivity residual for a real system in this repository. The partition-free referee reports its own block-additivity residual on a two-Gaussian model density (worst 1.735e-18 Ha, row S21), which is a different object and not interchangeable with this one. Earlier versions of this row cited the two json logs for the residual: they carry the per-system tables and no additivity field at all
S3	`nlcsplit/step_f_dispersion.py`	that $\beta\int\rho$ = βN cancels identically from a superposition difference, so the inter-fragment term needs no local-density convention — and the denominator rule that refuses to quote a share when the sum cancels
S5	`nlcsplit/step_e_kernel.py`, `evidence/recompute/20261001T083716Z_step_e_pairing_control.out`, `evidence/recompute/MANIFEST.sha256`	the definition-sensitivity control as both generator and record: the script compares same-centre with cross pairing on one grid and one density and prints the per-variant totals, and the archived stdout now carries the +0.1379/+0.4182/+0.1346 (level 1) and +0.9057 (benzene, level 0) kcal/mol of §2. Until 2026-10-01 this row pointed at a re-measurement only, because the stdout that recorded them lived in nlcsplit/logs, which is gitignored - and the unarchived benzene figure in this paper (0.914) did not match the archived one in the companion text (0.906); the run resolved it in favour of 0.906
S7	`nlcsplit/step_d_floor.py`, `evidence/recompute/20260928T150222Z_step_d_floor.out`	the `rho_floor = 1e-12` far-field cut, and the reported cost of it (10.7–22.6% of points dropped, carrying $\leq$ 2.3e-6 electrons)
S9	`nlcsplit/step_i_gridrepr.py`, `evidence/recompute/20260929T_step_i_gridrepr.out`	agreement of the explicit pair sum with PySCF's native VV10 kernel on a fixed grid, and the grid-representation residual quoted in §1 and §2
S10	`evidence/l2cp/20260928T213839Z_centos-step_h_l2cp.out`, `evidence/l2cp/20260929T051912Z_centos-l2_benzmeth_cp.out`	the level-2 counterpoise numbers, computed on the cluster and archived with their stdout
S12	`evidence/recompute/20260929T003144Z_stepg-recompute.out`	the basis-set and grid sensitivity of the attribution, including the ±5% agreement among the four basis sets beyond STO-3G
S17	`nlcsplit/tests/test_partition_factors.py`	the two assertions `partition_factors()` enforces before returning (pointwise normalisation at 1e-12, agreement with PySCF's own `get_partition` weights at 1e-11) — assertion thresholds, not measured residuals
S18	`evidence/orca_run/`, `evidence/orca_control/`, `evidence/orca_widen/`, `evidence/orca_widen/cross_program_table.txt`	the ORCA 6.0.1 outputs and their manifests (`evidence/orca_run/MANIFEST.sha256`, `evidence/orca_control/MANIFEST.sha256`, `evidence/orca_widen/MANIFEST.sha256`): six runs for the water and benzene-A anchors, the ωB97X control group, and eighteen more covering the remaining six systems, with the archived stdout for every one of the 24; plus the two generators that re-derive the comparison from those outputs and refuse if the system list or any row's definition disagrees
S19	`evidence/release_qa/`	clean-environment installs and out-of-tree runs, each recorded with the SHA-256 of the wheel that was installed

functionals. *Journal of Chemical Theory and Computation*, 20:49–67, 2024. doi: 10.1021/acs.jctc.3c00903.

[4] Predrag Lazić, Nicolae Atodiresei, Mojtaba Alaei, Vasile Caciuc, Stefan Blügel, and Radovan Brako. Junolo – jülich nonlocal code for parallel post-processing evaluation of vdw-df correlation energy. *Computer Physics Communications*, 181:371–379, 2010. doi: 10.1016/j.cpc.2009.09.016.